STACKLENS — MENTOR Q&A PREP SHEET
Team Delvrixo Syndicate · HackIndia Spark 4
==========
Share this with: Nitin, Vasu, Stuti, Nilesh
==========

SECTION 1 — FROM NILESH (Mentor Questions)

Q1. How can you earn money from StackLens?

- Free tier: 10 scans/month — builds user base, zero friction to try
- Pro plan: ₹999/month — unlimited scans, priority support, history
- API plan: ₹4 per scan — for agencies, tools, automation pipelines
- Enterprise: custom pricing — bulk URL analysis, white-label, SLA
- Comparable: BuiltWith Pro charges ₹24,000/year with fewer features
- Year 1 goal: 500 Pro users = ₹60 lakh ARR
- Revenue is recurring (subscription) — not one-time

Q2. How will you do marketing?

- Developer communities first: Reddit r/webdev, Dev.to, Hashnode, IndiaHacks
- Post on Twitter/X — "I analyzed stripe.com in 60 seconds, here's what I found"
  → goes viral organically with developers
- Product Hunt launch — developer tools consistently trend there
- YouTube shorts: screen recordings of analyzing famous sites
- College tech fests and developer bootcamps — direct student reach
- Partnerships with coding bootcamps like Newton School, Masai, GUVI
- Cold outreach to Indian startup founders (via LinkedIn)
- SEO: "how is [website] built" gets millions of searches monthly
- Zero paid ads needed initially — product demos itself

Q3. How can you scale this?

Phase 1 (Now): Local Ollama, demo-ready, HackIndia MVP
Phase 2 (3 months): Deploy on cloud VPS, add Razorpay payments, launch SaaS
Phase 3 (6 months): VS Code extension, Chrome extension, bulk API
Phase 4 (12 months): SDK for developers, white-label for agencies
Phase 5 (2 years): Enterprise contracts with IT firms, TCS/Wipro/Infosys

Technically:
- Replace Ollama with hosted Mistral API for cloud version
- Use Redis for caching — reduce repeat analysis load
- Add CDN for frontend (Vercel/Cloudflare)

- Horizontal scaling with Docker + Kubernetes for API

Q4. How is it different from other tools? Name them.

Wappalyzer:
- Browser extension only
- Detects ~30% of technologies
- No AI explanation, no code output
- Can't handle JS-heavy SPAs well

BuiltWith:
- Website and API
- Better detection but paid (₹24,000/year)
- No AI, no code skeleton, no architecture explanation
- Passive — doesn't explain WHY choices were made

WhatRuns:
- Chrome extension only
- Basic detection, no AI, no explanation

SimilarTech / Datanyze:
- Enterprise sales intelligence tools
- Not for developers, no code output

StackLens advantage:
✓ 4-layer fallback — always returns results
✓ AI explains WHY (not just what)
✓ Generates ready-to-use code skeleton
✓ Runs 100% locally — no data leaks
✓ Free for students
✓ Security hardened (6 measures implemented)

Q5. RAM usage and VPS deployment cost?

Current local setup:
- Mistral 7B model: ~4.5 GB RAM
- FastAPI backend: ~150 MB RAM
- Playwright (when active): ~300-500 MB RAM
- Total minimum: 6 GB RAM recommended, 8 GB comfortable

VPS recommendation for deployment:
- Remove local Ollama, use Mistral API instead (cloud inference)
- Then VPS requirement drops to: 2 GB RAM, 2 vCPU
- Cost: DigitalOcean/Hetzner — ₹1,200-2,000/month
- Or Render.com free tier for initial launch

With Ollama on VPS:
- Need minimum 8 GB RAM VPS
- Cost: ~₹5,000-8,000/month (not recommended initially)
- Better to use Groq API (free tier) or Together AI for cloud inference
- Switch to cloud AI = same results, 10x cheaper infrastructure

---

SECTION 2 — FROM NITIN (Your Own Questions)

Q1. What is your Problem Statement (PS)?

Developers, students, and founders waste hours trying to understand how websites are built — manually inspecting DevTools, reading minified code, and guessing frameworks. Existing tools detect only 30% of the stack and give zero explanation or code output.

One line: "There is no single tool that gives a complete, instant, AI-powered breakdown of any website's full technology architecture."

Q2. What is your solution?

StackLens — paste any URL, get:
- Full tech stack (1500+ technology signatures)
- Page architecture (component tree)
- AI-generated code skeleton (ready to use)
- AI explanation of WHY those technologies were chosen
- Performance metrics (load time, size, HTTPS, compression)
- UI pattern detection (buttons, modals, carousels)

All in under 60 seconds. Fully local. Zero API cost.

Q3. Who will use this and why? (India-specific)

Primary users (free tier):
- Engineering students learning web development
- Bootcamp graduates studying real-world stacks
- Hackathon participants doing competitive research

Secondary users (Pro ₹999/month):
- Freelance developers pitching to clients
- Startup founders doing competitor analysis
- Junior developers at agencies

Enterprise (custom):
- IT consulting firms (TCS, Wipro, Infosys vendor teams)

```
- Cybersecurity firms doing web recon
- Digital agencies managing 50+ client sites

Why India specifically:
- 27M+ developers, fastest growing developer market globally
- Students need free tools — we give 10 free scans/month
- Price-sensitive market — ₹999 vs $295 (BuiltWith) is 25x cheaper
- "Made in India, Priced for India" positioning


Q4. Future roadmap — SDK, extensions, API, SaaS?

- Chrome Extension: one-click analysis while browsing any site
- VS Code Extension: analyze competitor sites without leaving editor
- SDK: npm package — `stacklens.analyze(url)` for developers to embed
- Bulk API: analyze 100 URLs in one call — for agencies and scrapers
- SaaS: full cloud version with team collaboration, shared history
- Webhook integration: get notified when a competitor changes their stack
- LLM integration: plug our API into ChatGPT, Claude workflows


Q5. What security measures did YOU implement? (Nitin's section)

As the cybersecurity implementer on this team, I personally built:

1. SSRF Protection (Server-Side Request Forgery)
   - What it is: Attacker passes http://169.254.169.254 (AWS metadata)
     or http://localhost:8000/admin to steal internal server data
   - What I built: check_ssrf() function that blocks all private IPs
     (127.x, 192.168.x, 10.x, 172.16-31.x), link-local addresses,
     dangerous ports (3306, 6379, 27017), and file/ftp/data schemes
   - Runs before EVERY outbound request — twice per analysis

2. Rate Limiting
   - What it is: Attacker sends 10,000 requests to burn your server
   - What I built: SlowAPI middleware — 10 requests/minute,
     50 requests/hour per IP. Returns HTTP 429 if exceeded.

3. Firebase Token Authentication
   - What it is: Anyone who knows port 8000 bypasses your frontend
   - What I built: require_auth() dependency on all protected endpoints.
     Every request must include a valid Firebase ID token in the
     Authorization: Bearer header. Backend verifies it server-side.

4. XSS Prevention (Cross-Site Scripting)
   - What it is: Malicious website returns <script>steal(cookies)</script>
     in their HTML — if we render it raw, attacker owns the user's browser
   - What I built: Replaced dangerouslySetInnerHTML entirely with a safe
```

```
      React tokenizer (tokenizeLine function) that parses code into typed
      segments and renders them as plain React spans. Zero HTML injection.

5. Content Security Policy (CSP) Headers
   - What it is: Even if XSS gets through, CSP blocks script execution
   - What I built: Full CSP in next.config.js covering script-src,
     style-src, connect-src, frame-src (DENY), object-src (NONE).
     Also added X-Frame-Options: DENY, X-Content-Type-Options: nosniff,
     Referrer-Policy, Permissions-Policy.

6. Input Sanitization (Both Frontend + Backend)
   - Frontend: sanitize.ts strips null bytes, control characters,
     validates URL structure, blocks private IPs, rejects embedded
     credentials (http://user:pass@host), rejects javascript: URIs
   - Backend: Pydantic validator + URL parser + SSRF check — triple layer
   - Request size limit: 4KB max body — blocks oversized payloads

Most hackathon projects have zero security.
We have 6 production-grade measures implemented.


Q6. What technologies were used and WHY each one?

FRONTEND:
- Next.js 14 — React framework with SSR, file-based routing,
  built-in optimization. Chosen for fast initial load and SEO.
- TypeScript — type safety catches bugs at compile time, not runtime
- Tailwind CSS — utility classes, no custom CSS needed, fast to build
- Framer Motion — smooth animations, AnimatePresence for page transitions
- Lenis — buttery smooth scrolling, replaces browser default scroll
- Firebase Auth — production-grade auth in zero setup time

BACKEND:
- FastAPI (Python) — async Python API, auto-generates docs,
  Pydantic validation built-in, fastest Python framework
- Playwright — headless Chromium for full JS rendering (Layer 1).
  Only tool that accurately detects React, Next.js, Vue, Angular.
- BeautifulSoup + Requests — HTML parsing fallback (Layer 2)
- Ollama + Mistral 7B — local AI inference. Runs on your machine.
  Zero API cost. Zero data leaks. Works offline.
- SlowAPI — rate limiting middleware for FastAPI
- Firebase Admin — server-side token verification

DATABASE:
- Firebase Firestore — NoSQL, real-time, zero infrastructure setup.
  Used for analysis cache (30 min TTL) and history.
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
SECTION 3 — ADDITIONAL QUESTIONS (Predicted)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

Q. What if a site blocks your scraper?

- Layer 1 (Playwright) gets blocked → fall to Layer 2
- Layer 2 (Requests) gets blocked → fall to Layer 3
- Layer 3 (Headers only) gets blocked → fall to Layer 4
- Layer 4 (Ollama AI) always works — infers from domain knowledge
- We also have a known-sites database with 25 pre-loaded stacks
  (Stripe, Vercel, GitHub, TryHackMe etc.) for instant results
- We NEVER return a blank screen. Ever.


Q. How accurate is the AI?

- For known sites (our database): 100% accurate
- For sites with JS rendered content (Playwright): 85-95% accurate
- For sites that block scraping (AI domain inference): 60-75% accurate
- Rule-based detection always takes priority over AI guesses
- AI fills gaps, doesn't replace evidence-based detection


Q. Why not just use OpenAI API?

- Cost: GPT-4 costs ~₹6-8 per request. At scale = ₹60,000/month
- Privacy: User's competitor data goes to OpenAI's servers
- Dependency: If OpenAI goes down, tool goes down
- Latency: API calls add 2-5 seconds vs local inference
- Ollama Mistral 7B: ₹0 per request, local, always available


Q. How long did it take to build?

- Built in 36 hours at HackIndia Spark 4
- Team of 4: Nitin (backend + security), Vasu + Stuti (frontend/UI/UX),
  Nilesh (backend support + integration)
- All features functional: auth, analysis, history, export, security


Q. What's the biggest technical challenge?

- The 4-layer fallback system — making it seamlessly try each method
  without exposing failures to the user was the hardest part
- SSRF protection — needed to block private IPs without breaking
  legitimate sites that use non-standard configurations
- Making Ollama output consistent JSON — LLMs are unpredictable,
```

needed robust parsing with multiple fallback strategies

Q. Can this be used for malicious purposes?

- Any web analysis tool can theoretically be misused
- We require Firebase authentication — anonymous use blocked
- Rate limiting prevents bulk automated scanning
- SSRF protection means it cannot scan internal networks
- All scans are logged in Firestore for audit purposes
- We only analyze publicly accessible web pages —
  same information available to anyone with a browser

Q. What would you do with funding?

- ₹10 lakhs: Hire 1 backend dev, cloud deployment, marketing
- ₹50 lakhs: Full team, VS Code + Chrome extension, enterprise sales
- ₹1 crore: Scale to 10,000 users, API partnerships, international

---

SECTION 4 — TEAM ROLE DIVISION (for presentation)

NITIN RAJ — Team Leader
- Problem identification and solution architecture
- Backend development (FastAPI, scraper engine, AI engine)
- ALL security implementation (SSRF, rate limiting, auth, XSS, CSP, sanitization)
- Firebase integration (backend Admin SDK)
- Pitch and presentation

VASU — Frontend Developer
- Landing page UI and UX design
- Component architecture in Next.js
- Framer Motion animations and Lenis scroll
- Responsive layout and mobile optimization

STUTI — Frontend Developer
- Dashboard UI (analysis results, tabs, sidebar)
- Design system (colors, typography, brutalist theme)
- Auth pages (login, signup, forgot password)
- UI/UX polish and visual consistency

NILESH — Backend Support + Integration
- Backend integration support
- Frontend-backend API connection
- Testing and debugging

- Demo site pre-caching

QUICK REFERENCE — ONE LINERS

What is StackLens?
"Paste any URL. Get the full tech stack, architecture, AI explanation, and code skeleton — in 60 seconds. Free. Local. Secure."

Why not competitors?
"Wappalyzer gives logos. BuiltWith charges ₹24,000/year. Neither explains WHY, neither generates code, neither uses AI. We do all three — for free."

Why local AI?
"Zero cost. Zero data leaks. Works offline. No rate limits."

Security in one line?
"6 production-grade security measures. Most hackathon projects have zero."

Business model?
"Free for students. ₹999/month for pros. ₹4/scan for API. BuiltWith earns ₹2 crore/year doing less."